1 Overview

The *Software Bill of Materials* (SBOM) has recently gained heightened attention as a critical mechanism for securing the software supply chain. Often compared to a "nutrition facts" label [13], SBOM provides a structured, hierarchical inventory of every software component—including third-party libraries, open-source dependencies, and proprietary code—used within a given application [12]. This level of detail not only fosters transparency by revealing how complex, modular software systems are assembled but also enables organizations to conveniently assess and mitigate security vulnerabilities, licensing issues, and other risks. Notably, in contexts such as the Internet of Things (IoT) [101], where diverse and frequently updated components converge, maintaining an accurate SBOM is essential for comprehensive risk analysis and impact assessments. By cataloging critical metadata—such as component names, suppliers, versions, and authors—SBOMs empower stakeholders to make more informed decisions throughout the software lifecycle, thereby bolstering the resilience and accountability of modern software ecosystems.

Modern software is assembled using third-party (proprietary and open-source) components that are integrated to achieve the desired functionality. An accurate inventory of all components could enable organizations to identify vulnerabilities, implement rapid impact analysis, and support efficient responses. The *Software Bill of Materials* (SBOM) is designed to provide exactly that information necessary for risk-aware decision-making. It is a data artifact consisting of a machine-readable nested inventory of the components that make up a software product [75]. SBOMs can support the systematic identification of vulnerabilities, to ensure that software owners know that remediation is needed [90]. SBOMs have the potential to improve transparency throughout the supply chain and give stakeholders the crucial information to support decisions throughout a product's lifecycle, from procurement, to operations, to deprecation. SBOMs are focused on improving human decision-making by providing information about known vulnerabilities to software consumers. However, SBOMs currently identify every vulnerability in every component without determining if these are even potentially called in the codebase. The recommended mitigation by default is to patch and update dependencies, which may not be actionable for the application (e.g., due to downtime requirements).

Difficulties exist in maintaining accurate and up-to-date SBOM for reasons including incomplete data, naming inconsistency, and the execution paths of the code. Inconsistencies across SBOM formats and tooling ecosystems also hinder unified SBOM deployment. Organizational hurdles, such as awareness, training, and resource allocation are only beginning to be addressed. In addition, there has been very little research on the usability of the tools designed to translate the data into human-centered decision support.

As noted, SBOMs also provide information about every possible dependency (defined in a very broad sense). The lack of accuracy in terms of actual exposure to vulnerabilities results in high levels of false positives. As a result, SBOMs currently risk being inaccurate, inactionable, and ignored.

SBOMs have the potential to provide better risk management and support informed decision-making based on comprehensive data about dependencies. SBOMs can systematically impact the awareness of vulnerabilities and risks in software development. SBOMs could support actionable, timely, usable risk management based on dependency information and data in the larger vulnerability ecosystem. With effective risk communication, SBOMs can support risk-aware decision-making as code is authored and altered. Accurate, efficient, and usable SBOMs are arguably a precondition for their rapid and effective deployment.

In terms of usability, the complexity of SBOM data often makes it challenging for developers and stakeholders to interpret and act upon the information effectively. In human subjects research we found that participants dropped out of every experimental group even when provided with popular data visualizations [67,69]. The more usable systems, such as the Linux Zephyr product, treat the SBOM as an output, not an on-going decision support system.

Our goal is to provide agnostic code analysis tools that can be used at any point and by any development platform and by any user, requiring only the existence of the code to produce accurate, actionable, and usable information.

2 Intellectual Merit

The research on how vulnerabilities arise in development practice and how this might be mitigated informs both dynamic and static analysis and usable security in this cross-domain integrated proposal. Providing detailed results of analysis is useful only to the degree that the results are understandable, actionable, and thus usable. These are usable only if they are accurate.

In addition to the specific application to creating accurate SBOMs, our insights may contribute to more general software engineering tasks. For example, code smells [63] have largely been used to drive refactoring efforts. However, as the above suggests, since code smells indicate code that 1) is likely to be significantly modified in the future (which may introduce bugs or change behavior) and 2) is perhaps more likely to contain bugs due to its poor quality, the value of smells could be improved by allowing developers to understand when their current code relies on potentially poor-quality code and limit such dependence. We also speculate that, due to their general ability to perform software engineering tasks focusing on correctness or fault detection [31, 84, 104], LLMs and more generally AI-based assistants may soon offer a kind of static analysis that is closer to a general notion of *marked code* (see below) than traditional analyses. In essence, code analysis should be about more than paths, and information used by humans to make decisions about how to change code (e.g., to mitigate vulnerabilities) needs to combine well with dynamic and static analyses of various degrees of precision and with various meanings; this project aims to bring together such a synergy, with the focus of making SBOMs secure, accurate, and usable.

2.1 PI Qualifications

PI Groce has extensive experience in coordination of complex research efforts focused on release of open-source tools. He coordinates efforts for the DeepState project, and has managed releases of the Echidna smart contract fuzzer (both tools with more than 800 GitHub stars, dozens of forks, and 20 or more contributors). He has long experience with static and dynamic analysis of critical code, including in a security-centric context and for code at NASA/JPL controlled by complex change and review standards. He has worked with Trail of Bits, a leading industrial research and practice firm in the cybersecurity sector, to develop the state of the art in smart contract static (and dynamic) analysis tools. Key concerns in smart contracts include information access and complex dependency analysis. Groce's work more largely concerns advanced code analysis tools (dynamic and static) usable by non-expert developers, including research scientists in non-CS fields, published in over 150 papers with more than 8,000 citations to date.

Co-PI Camp has extensive experience in human subjects research evaluating the interactions of human and machine behaviors. Her contributions (beyond peer reviewed works) include responses to Federal RFCs, policy white papers, tools for interaction design including developer support tools, and sanitized datasets. Her approaches to safety include anomaly detection by combining out-of-band information (e.g., layoffs, macroeconomics) with network data, field studies, laboratory observations of tools with developers, and controlled experiments. Camp has been PI and managed $20M in grants from Federal (including the National Science Foundation, DARPA, DHS), private sector (including Google, Cisco, Microsoft), and non-governmental sources (including ARIN and CTIA). Her most recent collaboration was with the Department of Homeland Security, building on research on the Cyber Trust Mark and the Software Bill of Materials.

2.2 Attention/Relevance/*Causal* Analysis

In order to provide risk mitigation it is necessary to systematically, reproducibly, and reliably identify *relevance*. *Static* analyses always face a trade-off between being over-sensitive (over-approximating behaviors of a program) and being under-sensitive (under-approximating behaviors; the later is also a key limit of dynamic analyses). Human attention is always a critical resource in code analyses [10]; in our case the determination of where to focus user attention is *the entire purpose* of the static and dynamic analysis: using SBOM as the end purposed to guide human determination of actions to be taken and estimation of risks. Most dynamic and

static analyses in the literature are essentially organized around a concept of an error, typically instantiated in, e.g., a (possible) path, that indicates a potential flaw in the code.

The advisory aspect of all analyses is fundamentally altered in the case of actionable SBOMs. Static analysis systems often simply indicate the presence of a path possibly leading to dangerous runtime behavior; many tools also offer advice on correction of such behaviors. Such advice, however, is almost always limited to proposing a *semantically almost equivalent* behavior. The behavior is technically not semantically equivalent, of course, in that it is intended to remove the potential for erroneous behavior, but in practice most suggested fixes do not change the larger intent of a program. Even advice to change functionality to be more secure (e.g., invoke input sanitization) does not usually aim to prevent intended uses of a program, but to frustrate adversaries who would exploit a fixed functionality. Such an approach does not advise users as to when *removal of some functionality* may achieve the same goal, even when such changes are user-facing, and advise as to how much impact such a removal can be expected to have. However, in the SBOM case, such is the basic use-case: code beyond your control is vulnerable, perhaps in unknown ways.

The core problem can be stated in a very general way: First, a method (static or dynamic) *marks* a function, file, module, or entire library. The meaning of a mark may vary widely. It may mean, for example, including sensitive or private information relevant to a regulation such as GDPR. It may mean having poor coding practices that lead to higher likelihood of having errors (smells, as above). It may mean modification by a developer flagged as posing a security risk (see below). And, of course, most importantly, a mark may indicate that a vulnerability is now known to be present at this code location. The key feature of such a mark is that it is *not* a single path through a program, and thus amenable to traditional analysis of paths through calling code. It is best thought of as simply indicating a need for caution or further analysis. This does not mean that a mark has no structure; it may mark certain code features (e.g., functions, data types or variables, or code blocks) as relevant to the establishment of the mark, and others as explicitly not marked. The *first problem* is to establish which other code, *on the basis of its connection to marked code,* should also be designated as requiring attention. An additional problem, to limit the expansion of needed attention, is to suggest ways to modify code so that it no longer needs such a designation. In the limit, this means not importing or calling the marked code, but also includes *limiting* the interaction with marked code.

The utility of such analysis can perhaps best be understood in light of the infamous supply chain attack on sshd discovered in April of 2024 [6]. The primary attack was carried out by a GitHub account owned by a (likely non-existent) Jia Tan. This Jia Tan made over 5,000 code changes to at least seven projects over the lifetime of that GitHub account [43]. Even with the level of attention this attack immediately received, analyzing all of the changed code and determining any subtle risks associated would have taken substantial time. Some changes "Tan" made included replacing a safe function with a less memory-safe function, albeit in a context where this *does not* appear to induce a vulnerability. Developers wanted, as soon as possible, to know when their code interacted with code modified by such a suspicious developer, even when no normal static analysis warning was produced, and no dynamic behavior was inherently suspicious. In short, developers wanted to know about *causal flow* from Tan's edits to their own code.

2.3 Human Dimensions of SBOMS

The difficulty of working with SBOMs has been recognized. The Linux Foundation developed training materials for the software package data exchange (SPDX) SBOM generator [91]. Similarly, the CycloneDX working group has training resources [21] for its own generators, explaining how to create and leverage SBOMs. Beyond training, multiple visualization tools have been developed. The challenge with raw text SBOM files is that text-based SBOMs in formats like JSON or SPDX can be overwhelming and difficult to interpret. It is not easy to identify relationships between components or understand the overall structure. Visualization is arguably necessary for SBOM data, as complex data is more easily understood visually, enhancing both accuracy and usability.

SBOM2dot [48], ItDepends from TrailofBits [92], Deepbits [55], and Guac (Graph for Understanding

Artifact Composition) [1] from Google all use the same model of graphical representations with the option of deeper exploration. Yet current research in usability is lacking. The comparisons by [67,69] documented that even with decision-making focused on the graphical tools developers systematically fail to identify vulnerabilities, can not consistently identify mitigations, and underestimate the risk of those vulnerabilities they do identify. An exploratory conversational AI approved these results; yet this remains a stand-alone product that would not necessarily be adopted by developers. In the proposed research, we will examine how to integrate risk communication and SBOM data into the coding and management processes.

2.4 Log4Shell as an Example

Currently it is possible to search for vulnerabilities using the National Vulnerabilities data (NVD) based on a measure of severity from the Common Vulnerabilities and Exposure (CVE) measures that are based on the Common Vulnerability Scoring System (CVSS). These may appear mechanical and deterministic, yet research has shown that such risk measures are not at all consistent [102]. Their relatively low fidelity and lack of consistency has not made prevented them from being widely used. This landscape informs our proposed research on vulnerability mitigation decision-making. There is not a database of APIs, tokens, or code to support risk-averse decision making; nor a generally applicable way of generating these.

An example of the complexity of vulnerability management can found in the Log4j critical vulnerability and exposure (CVE) of 2021, dubbed Log4Shell [9, 74]. Log4j is a popular open-source logging framework, maintained through the Apache Software Foundation, that is widely used in software performance and security event tracking modules in proprietary and open-source software products across the software industry [19, 38, 39]. The critical remote code execution (RCE) vulnerability, CVE-2021-44228, in versions of Apache Log4j software libraries is attributed to having been a major component to a 34% increase in global software vulnerability exploitations between 2020 and 2021 [54].

An explanation for the jump in vulnerability exploitations, observed between 2020 and 2021, may lie in the sheer volumes of usage of Log4j libraries. A compounding or alternative factor may also rest in its widespread use as a hub dependency in a vast number of different packages across the software ecosystem. The broad use of Log4j across the software industry [19], and the vulnerability in its versions may have therefore had an over-sized effect. Many people exposed to risk did not know that their own code was vulnerable.

The CVSS v4.0 Base score for the Log4j vulnerability of 2021 (CVE-2021-44228), was 10 - the highest possible score on the 0.0 to 10.0 scale, where scores 0.1 to 3.9 are rated 'Low' severity; scores 4.0 to 6.9 - 'Medium'; 7.0 to 8.9 - 'High'; and 9.0 to 10.0 - 'Critical'. Numerous other vulnerabilities discovered in 2021 had a base score of 10.0 however [20], including a vulnerability in the Metabase open source data analytics platform (CVE-2021-41277); an authentication bypass vulnerability in Zoho ManageEngine ServiceDesk (CVE-2021-44515), and a remote command execution vulnerability in Sitecore XP 7.5 (CVE-2021-42237). None of these had the same broad impact as the Log4j vulnerability, however, supporting the observation that vulnerabilities accorded *the same CVSS severity score may have widely different acuteness at broad-scale and at population level*.

Beyond improvements in secure coding, our approaches to identification of risk and impact using the observed behavior of code can better assist organizations to distinguish the impact of a vulnerability beyond simple CVSS rating.

2.4.1 Usability, Acceptability, and Accuracy of Security Tools

In order for SBOM (and CVSS) to influence decision-making the information must be presented in a manner that is usable, actionable, and not burdensome. The presentation of vulnerability information obtained by combining vulnerability identification with dynamic and static analysis of relevancy will be informed by previous analysis of security tools.

Usability frameworks, such as Nielsen's heuristics [77] and ISO standards [3], provide basic principles for designing and evaluating security and risk assessment tools. Jones and Tate leveraged these design principles in their theoretical analysis of changes in SBOM visualizations over time [59]. They noted challenges to usability in different approaches to graphing SBOM data, providing illustrative examples but not measurements. They also concluded, as has been the choice in our analysis of SBOM visualization, that representing SBOM data as a graph is the obvious choice. Not only can such representations align with human cognition, but also such visualizations can also enable comparison of applications using or containing different languages. This further motivated our choice of graphing tools, as it is increasingly common for modern software applications to include multiple programming languages [7, 64, 65, 79].

Technology acceptance models (TAM) [26], and unified theories (UTAUT) [100] are frequently used tools to analyze the acceptability of a system. Due to the highly quantifiable nature of our task and choice of a survey for an A/B comparison, we selected the NASA TLX index (as described in Section 4.2). Case studies have demonstrated that inadequate user acceptance often decreases the effective integration of innovations into existing workflows [22]. Integration of SBOMs into existing workflows is not a solved problem, so other adoption of other cases (e.g., [72]) informed the design of our experiment. The first pilot of SBOM was focused on health care [49] and the lack of usability and the difficulty of matching the correct BOM to the correct code was a significant obstacle to use.

Additionally, accuracy in SBOM data is important for ensuring the visualized vulnerability discovery and mitigating process. Empirical research has shown that even minor inaccuracies can significantly change risk perception, decision-making speed, and error rates [94]. Therefore, careful evaluation of SBOM visualization tools, calibration of SBOM graph details, and comprehensive analysis of usability, acceptability, and accuracy are crucial usability dimensions in advancing the efficacy and reliability of SBOM visualization tools in real-world security contexts.

Visual representations play a critical role in clarifying the complex dependency graphs inherent in SBOM. Recall the functional purpose of SBOM is to enable stakeholders — including developers, security analysts, product owners, and government — to rapidly identify and address potential vulnerabilities [14]. Various organizations are starting to adopt the SBOM visualization tools, including SBOM2dot [48], ItDepends from TrailofBits [92], Deepbits [55], and Guac (Graph for Understanding Artifact Composition) [1] from Google. Common visualization techniques include force-directed graphs, hierarchical trees, and tabular dashboards, each offering different advantages in SBOM visual clarity. For example, force-directed graphs from ItDepends provide a structured overview; however, this can be overwhelming with nodes and dependencies when the software contains significant depth and complexity. Despite these approaches, challenges persist regarding scalability, user interface design, and data size, all of which can hinder users' capacity to effectively interpret and act on risks. Moreover, there is a need for empirical studies examining how SBOM visualizations affect security-related decision-making. This has been identified as an important research gap, particularly in the role of SBOMs in communicating vulnerability information [99]. Jones and Tate identified graphs as the natural method of providing SBOM information, providing examples but not analyzing efficacy nor usability [59].

2.5 Security Errors in Coding

One goal of the proposed research is to make writing code without known vulnerabilities tractable. There is a dearth of usable security research on SBOM, preliminary work by the PIs can be found in Section 4.2. Simple errors in development have long been understood as a security challenge.

An early call for understanding the requirements of developers provided a set of ten principles for creating secure and usable APIs, focusing on the high cost of flawed cryptographic APIs [42]. They advocate for aligning APIs with pre-existing perceptions and recognizing the limits of the average developer's understanding of cryptography. Just as cryptographic APIs embed an expectation of high levels of security expertise, privacy requirements may depend upon developers' understanding of the compliance risks of all

the contexts in which code may be used or the risks of a particular snippet or dependency in their targeted context.

Our research is further motivated by Acar et al.'s work [4] that calls for a research agenda for developers. This research is aligned with the call for a better understanding of both developers' knowledge and the status quo. Specifically, we seek to better understand developers' security and vulnerability knowledge and measure the status quo by evaluating the interaction of human subject participants with code snippets obtained from popular GitHub repositories. As with previous work, we used GitHub as a sample for code; however, our recruitment of developers was less targeted [5]. In a study close to the human-subjects research on developers, Chamila and Nalin conducted a qualitative study with 40 software developers. They revealed that most programmers believe they should be responsible for end-user security [98]. Yet this does not result in secure code, nor do the developers report following best practices. Their lack of security expertise and the extra time required for testing were two main reasons for not following the secure development standards. Our results indicate a similar lack of privacy expertise [34].

Previous analysis of GitHub has shown that security practices are lacking, sometimes in obvious ways. An analysis of coding practices on GitHub has been done by Yasemin Acar et al. They recruited 307 active Github users to complete the same security-relevant programming tasks, and they found being a professional did not increase a participant's likelihood of writing functional or secure code statistically significantly [5]. Another work on security issues about Github examined billions of files including real-time public commits and a snapshot covering 13% of open-source repos. This situation continuously exposes both developers and services to heightened risks of security breaches and potential misuse [70]. Besides the risks caused by the programmers, the researchers found chronic exposure to authenticating information, such as keys and passwords [85, 97]. The authors found authenticating data in public repositories along with their variable names, including 'pwd', and 'password'.

Lack of reliable sources and inefficient sources have been identified as drivers for such insecure practices. An examination of the manner in which developers search for coding examples found that peer-based systems are a source of failures in practice [4]. In a comparison of four experimental groups of developers asked to implement security functions, those using Stackoverflow were found to be most likely to provide insecure implementations. Those using formal company documentation were most correct in terms of secure implementations but also most likely not to complete the tasks. A follow-up focused on misconfiguration and other security errors in Android found that 15.4% of a 1.3 million sample contained security-related code snippets from Stack Overflow. Out of these 97.9% contain at least one insecure code snippet [37]. The most widely used snippets included security warnings but were nonetheless included. Similarly, in our selection of snippets from GitHub, we focused on more popular and widely used code.

The existence of nudges and sample code has proven effective in altering the security and privacy choices of developers in other domains [89]. Thus development and diffusion of usable, clear, secure examples are components of our research, and potentially a contribution in themselves. Developers cannot simply rely on a "bare bones" SBOM and advice from Internet forums or uninformed general-purpose chatbot queries to drive successful vulnerability mitigation responses.

3 Preliminary Work

The proposal is a for a new collaboration between established researchers in usable security and code analysis. In this section we provide an overview of our initial shared data source based on [16]. We then describe the methods for developing accurate, reliable, responsive SBOMs and the interactions in which these will be embedded. The Collaboration Plan makes clear that our efforts and teams will be integrated. While the usability and code analysis methods are introduced discretely, there will be regular interactions. The focus of the code analysis will be informed by the usability results in order to focus on those components that are most used and most usable. The usable security component will be informed by the scope, scale, and depth of the code analysis.

3.1 Data Compilation and Availability

Quantification of the implications of the potential harm of any vulnerability in code is challenging. The challenge of identifying exposure of a particular system to a given vulnerability is a significant driver of the Software Bill of Materials (SBOM). Metrics like the Common Vulnerability Scoring System (CVSS) quantify severity of vulnerabilities at individual or organizational levels, they do not extend to include components or measures for severity in wider contexts [36]. CVSS scores are nevertheless widely appropriated as core metrics for assessing vulnerability severities at broad scale, and by proxy, the risks posed by the vulnerabilities in greater contexts. They are therefore commonly used as primary measures for use when prioritizing risk mitigation of vulnerabilities [56]. This leaves a gap in metrics for appropriately assessing and prioritizing mitigation of vulnerabilities at population level.

A central goal of the proposed work is that it is generalizable. To compile the sample of software dependencies for the general pool of software, we generated SBOMs from the source code of over 100,000 randomly selected public GitHub repositories, by exporting their dependency graphs using the the GitHub API. The dependency graphs for GitHub repositories show dependencies that the ecosystems and packages in the repositories depend on, and packages that depend on the repositories. They are generated from dependencies submitted to the repositories using GitHub's dependency submission API, and from manifest and lock files stored in the repositories [41]. As of this writing the archive contains the SBOM of 145,081 repos.

The dependency graph in GitHub identifies upstream dependencies, and downstream dependencies in publicly available software repositories. These may include binaries required during the software build process, as well as those utilized at runtime during program execution. The graph includes metadata on each dependency, including information on known security vulnerabilities [41,82]. For the current compilation of over 145,000 repos 19% of all packages depend on *debug* (with 9% of all repos depending on a single version) and *glob* is at 17% . This is a result of the hosting environment rather than coming in from the code itself. To generate the dependency graph, GitHub parses a repository's manifest and lock files, which explicitly declare software dependencies. Manifest files define the metadata and dependency specifications for a given project or package, while lock files document the resolved versions of direct and transitive dependencies at the time of installation.

When enabled, GitHub can infer transitive dependencies from the project's direct dependencies using insights drawn from its extensive ecosystem, which comprises millions of open-source repositories. However, inferred transitive dependencies do not include specific version information. This inference-based approach contrasts with other methods that identify dependencies by analyzing build artifacts or packages upon their publication to a software registry. The GitHub dependency graph is enabled by default for public repositories and can be manually activated for private repositories [41].

We used python scripts running on Google's Colab platform to make the API calls for generating the SBOMs. The scripts used a pseudorandom number generator to select repository IDs for the SBOM extractions. Exported SBOMs were in SPDX-compatible JSON format, and they were programmatically written to folders in Google Drive. The SBOMs extractions in the public archive were performed daily between February 8, 2025, and May 2, 2025, and any SBOM generation attempts that failed —indicated by HTTP status codes 401, 403, 404, 500, or no response to the API calls — were excluded from the final dataset. The extraction continues and we expect to add an additional 100,000 in December.

The dataset is available on Zenodo.org [15].

4 Research Plan

4.1 Dynamic and Static Analysis for Nuanced Software Bills of Materials

Almost all current SBOMs describe components used by software at the package level; to a first approximation, the SBOM for a software component A enumerates all components $B, C \ldots Z$ *imported* by A. Zhao

et al. in their CovSBOM work [105] introduced a preliminary approach to address the gap (earlier noted by Stalnaker et al. [87]) between (1) the fact that a system A *may*, in the sense that it is clearly *possible* for A to call code from B (since B is directly or indirectly imported), be impacted by security or cryptographic weaknesses in a component B and (2) the real question of whether a particular weakness in B can *actually* impact A. CovSBOM addresses this with a lightweight, conservative, static analysis to determine which components (up to line granularity) in B *cannot* can be reached in practice from the code in A. However, the limits of their conservative static analysis are clear when an attention-relevance model is considered. The approach relies on a coarse recursive dependency graph construction. The most obvious limitation of this is that it does not even consider parameter values to function calls, such that if a function in A calls $f(\ldots, mode)$ in B, with a static parameter value for $mode$ that determines the control flow in f cannot call g, g will be marked as reachable from A. The relevance of this to usage of cryptographic libraries is obvious, since parameter values often determine the actual algorithms and configuration used. While a more sophisticated static approach could in theory address this particular problem in many instances, in general static analysis across function boundaries, much less software package boundaries, will over-approximate the reachable behavior of B from A.

An alternative approach for any A with a high-quality test suite is to dynamically estimate the coverage of B. This can be as simple as marking code elements (lines, functions, modules/files) as covered by the tests for A or not, of course, as in traditional software testing, and made more fine-grained by considering elements larger than a single line (whose coverage is usually binary by nature) more covered when higher percentages of their sub-elements are also covered. Subject to the limitation that A's test suite may not include all security-relevant behaviors of A, this can provide more information to use in the triage of vulnerabilities via SBOM. However, the software testing and analysis community has long observed that code executed in a test may not, in fact, be relevant to the disposition of the test. Consider the case where A calls a complex initialization routine in B that includes a faulty default value that is at the causal heart of a vulnerability. If A never uses any functions that reference the misinitialized value (perhaps it is for a cryptographic mode A is not using, if B is a cryptographic library), or even if A does use code in B that refers to the bad value, but always overrides the default value, then the true relevance of the vulnerability to A is either minimal or nonexistent. We propose to go beyond traditional code coverage to enable the discovery of such code, where coverage does not mean there is a causal path from a vulnerability to A's behavior.

4.1.1 Dynamic Relevance Precision: Hierarchical Mutation Analysis

The imprecision of static analysis in such cases suggests the use of dynamic analysis; important software systems tend to have large, high-coverage unit, integration, and regression test suites capturing the actual behavior of the system, and capable of determining when it deviates from correctness. Such test suites, though often incapable of detecting a vulnerability (due to it only arising in very unusual circumstances not captured by normal test efforts), are likely to be capable of determining *when A depends on a particular functionality of B under some circumstances*. Causal paths from B to A are by definition almost always much more common than vulnerability-inducing paths, and should be present in any adequate test suite for A. Simple code coverage is the easiest way, as stated, to observe such paths, but has some of the imprecision of static analysis, as the initialization example proposed above demonstrates. Fortunately, mutation analysis [29,78], originally developed as a way to evaluate test suites via the injection of artificial bugs, provides a *causal* approach to determine if code is relevant to A, given an adequate test suite. If a code element of B can be *completely removed* from B, and the test suite for A still passes, then A is unlikely to be able to observe vulnerabilities in that element. In traditional mutation analysis, this coarse-grained kind of mutation is usually limited to statement deletion, making the statement the only code element for which such relevance can be computed. This suffices for our initialization example, and in itself offers an advance simple coverage dependency.

However, it is also highly inefficient. For a large software system A, storing line-granularity dependency information for all used packages $B, C, D, \ldots$ would result in a very large SBOM, and require running the

test suite for A a number of times proportional to the total number of lines of code in all imported packages. Previous work on reducing the size of failing test cases (to establish causality and ease debugging) [73] and produce limited-functionality adaptations of software systems (by PI Groce as part of DARPA's BRASS [53] program) [18] leverages the hierarchical structure of systems to reduce this cost. A software package B is a hierarchical, tree-like ordering of directories, files, and functions; the tree that is B can be explored in a systematic top-down way such that when entire branches of the tree are not relevant to A this can be discovered with one execution of A's test suite. Given the frequency with which large multifarious packages are imported only to make use of one module or even function, this can produce a massive reduction in the cost of computing a precise SBOM.

The technical details of such subtree-removal are substantially different than in previous approaches; we wish to avoid difficulties in compiling subsets of B, so removing a file consists of replacing all functions/methods in a file with *NOP* versions that do not have any effect on program state. Functions that return values are modified to return a meaningless default value. Removing a directory is then simulated by removing all functions in all files in that directory and its descendants in the tree. In the case of C code, replacing a single function of type `T` with a compiling but non-functional stub is remarkably simple, due to C's facility with low-level type punning. A stub for `f` of non-void type is implemented as:

```
T f(params...) {
   T retval = {0};
   return retval;
}
```

Similar operations can be defined (with more or less difficulty) for all programming languages of which we are aware; the resource-adaptation work and work on pseudotested methods [96] introduce similar functionality for Java, for example. This allows us to make a complex, fine-grained estimation of relevance of a vulnerability, ranging across a variety of increasingly concerning scenarios:

1. The code in question is in the SBOM, but conservative static analysis shows it cannot be called.
2. The vulnerable code cannot be shown unrechable, but is never covered by any tests of A.
3. The vulnerable code is covered by some tests of A, but replacing the code with a NOP does not cause any tests of A to fail: A does not seem to observe the behavior of the code.
4. The vulnerable code is covered by A and changing the behavior of the code causes tests in A to fail.

The third relevance level can be made more precise by indicating how widespread A's test's indifference to the code is: (3.3) the vulnerable code and enclosing complete larger code units (files, up to directories, or large directory trees) without A being able to detect this change, (3.2) the vulnerable code can be removed, as can the function in which it appears, but the file in which it appears cannot be removed without tests of A failing or (3.1) the exact vulnerability can be removed, but the enclosing function cannot be removed.

While computing these relevancy levels can be fully automated, presenting such a hierarchy to users in an actionable, comprehensible way will require investigation into the human aspects of the *use* of SBOM-like information that makes up the second thrust of this overall proposal. The situation whose essence is presented graphically in Figure 1 may differ widely in details. It is not to be expected that those wishing to understand and make use of such information will know the details of even a system's test suite, much less the subtle notions of coverage vs. levels of mutation-based relevancy. The same basic situation in the graphic applies in cases where A *only* calls f, which has only one call, to g, and depends on no other values computed by B, C, or D (e.g. with statically *pure* functions f and g), or to cases where A calls f with parameters that are not statically computable, and because of those parameter values, the path through f

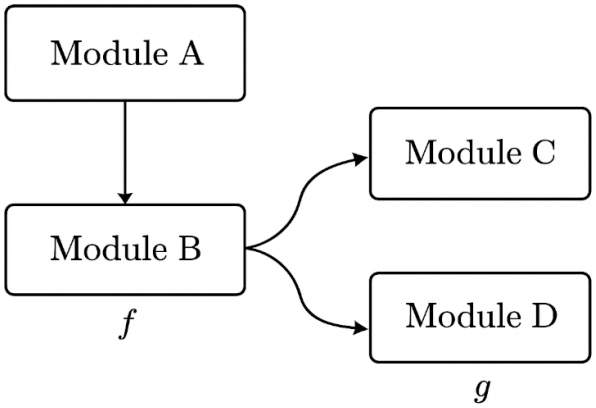

Figure 1: Dependency analysis of imports may widely differ from actual causal behavior. Module A imports Module B which imports modules C and D. Module A only uses function f from B, which uses only function g. How can users best understand the degrees of confidence in such divergences, and act on them?

avoids calls to other functions in B, C, D, or even where f in fact calls such functions, and their values have data-flow to the return from f, but A discards these values (e.g., they are part of a tuple-type return value and A only uses one part of the tuple). How can users make use of such rich-valued notions of dependence in practical settings is a core problem to be investigated in co-PI Camp's efforts in this proposal, and the two thrusts will inevitably iterate on what information to compute based on how it can be made actionable.

4.1.2 Using Relevance to Estimate Feasibility of Automatic Mitigation of Vulnerabilities

The DARPA BRASS work on adapting software to limited-resource environments is also applicable to removing a dependence on vulnerable code. However, resource-adaptation via code removal relied on manually-labeled test suites; dynamic relevance analysis can avoid this problem by informing developers when they can mitigate an unpatched vulnerability by modifying their own software. The lower the value on the above relevance-ranking, the more likely mitigation by removing the ability of A to call the code in B (thus changing the relevance to 1: code statically shown to be unreachable) is to succeed with little loss of functionality. The analysis also shows, when functionality must be sacrificed, *which* tests specify said functionality, allowing developers to assess importance (essentially our analysis automatically performs the labeling required by the BRASS approach, which is too onerous for rapid response to vulnerabilities).

4.1.3 Relevance Within Components

Our primary focus on relevance analysis in this approach to SBOM refinement, or in the discussion above of general uses of relevance-based analysis on relevance across software systems should not be read as meaning that relevance *within* a given system A is not also important. Traditionally, static analysis has greatly limited the degree of inter-procedural analysis due to the high cost of such analysis. Whole-program analysis is, in practice, extremely rarely performed, and procedure summaries [11] by nature lose important information about path feasibility in order to avoid the cost of analyzing flows across procedure boundaries. While whole program analysis is seldom practical, with fine-grained relevance information gathered during mutation analysis (or via LLMs) it should be possible to identify cases in which the cost of more substantial interprocedural analysis is warranted. Furthermore, unlike cross-component analysis for SBOMs, analysis within a program 1) should be computable with high precision based on the results of mutation testing of a single project only and 2) should be much easier for LLMs to estimate (for one thing, LLM-based predictive mutation testing techniques [57] can directly and very quickly estimate dynamic measures in this one-program case). Mutation testing is becoming much more widely adopted in industry and practice, making an advance in static

analysis that leverages this dynamic analysis highly attractive, since mutation analysis of important programs is likely to be carried out in any case. Projects benefiting from the testing and debugging improvement offered by mutation could, at no additional cost, benefit from more sophisticated static analysis.

4.2 Usable SBOMs

There are two major open usability questions in this research. One is the efficacy of classic risk communication for developers to assist in identification and ordering of risk, leveraging SBOMs. Click-thru graphical representations of dependencies fall into this category. The second is the possibility of designing novel risk communication tools from audio or video annotation to modern chatbot assistants powered by large language models.

• We will quantitatively measure the usability, acceptability and accuracy of SBOM format files, popular visualization tools, and different instantiations of AI interactions.

• We will explore developer perspectives on different presentations including qualitatively conducting a comprehensive open-coding of open-ended answers.

• We will create experiments to observe and document developer decision-making while coding when given different SBOM interactions.

• We will build an experimental environments to observe and document security analysts' decision-making when given different SBOM interactions.

• We will iteratively design tools with the overall goal of improving developer and security analysts' decision-making by providing usable, accurate information based on SBOM data.

In previous work we compared two SBOM visualization tools—*It-Depends* and *DeepBits*—against a machine-readable JSON file generated according to the SPDX standard [86]. *It-Depends* highlights dependencies and flags vulnerable packages, although it does not provide detailed metadata such as vulnerability source information [92]. By contrast, *DeepBits*, a commercial AI-based SBOM generation suite offers additional detail on package vulnerabilities, including *Common Vulnerabilities and Exposures* (CVE) identifiers. To serve as a control group, we employed JSON files adhering to the SPDX specification, which offers a more traditional, machine-readable SBOM format. We generated JSON-format SBOMs for three widely used Python packages (*Accelerate*, *Graphtage*, and *It-Depends*) under the SPDX standard. Participants were randomly assigned to one of three groups (i.e., *It-Depends*, *DeepBits*, or JSON) and asked to determine the presence of any vulnerabilities, identify relevant dependencies, and specify possible mitigation steps.

Two visualization tools, *It-Depends* and *DeepBits*, were selected for their capacity to generate clear, node-linked SBOM representations from JSON inputs. We conducted a randomized and controlled experiment in which participants interacted with SBOM visualizations exhibiting varying levels of complexity. The effectiveness of each visualization was evaluated on three dimensions: usability, acceptability, and accuracy in identifying vulnerabilities and proposing mitigation strategies. Usability was measured using the *NASA Task Load Index* (TLX), which assesses mental demand, physical demand, temporal demand, perceived performance, effort, and frustration. Acceptability and accuracy were gauged via user surveys and task completion rates.

We will use the same method in the first step of analysis in order to be able to compare results across systems and cohorts for the duration of the project. In the previous work, participants were given code scripts and corresponding SBOM graphs or JSON-format script files. The primary task involved analyzing the SBOM entries or graph visualizations to detect potentially vulnerable packages. After identifying vulnerabilities, participants either propose their own solutions or select from a pre-defined set of options to effectively address the security risks. We propose to expand this to observations of coding tasks, in the mode of We will then additional test the SBOM information provision to security analysts.

The measure of efficacy and acceptability was based on the assigned tasks in the first evaluation. First, participants were asked to determine which packages, if any, exhibited vulnerabilities, noting specific details about the nature and severity of these risks. Using these determinations, participants were asked to suggest

practical solutions for each vulnerability. They were asked about the risk of each vulnerability, then queried about usability.

In each condition, they were sequentially presented with several code components, and, for each component, asked to identify any vulnerabilities and the associated dependencies. If a component contained vulnerabilities, participants were further prompted to specify the nature of these vulnerabilities and propose mitigation strategies. To ensure consistency and feasibility, each participant was given a "one-click" link that provided descriptions and suggested mitigations for all vulnerabilities included in the study. These will be provided by automated analysis, as their importance was shown in this previous work.

The results varied significantly across the groups. In the *It-Depends* group, 25 of the 30 participants (83%) completed all assigned tasks. The *DeepBits* group included 35 participants, of whom 28 (80%) fully finished. By contrast, 47 participants were allocated to the *JSON* condition; however, only 23 (48.9%) successfully finished all tasks. BOMbot resulted in substantive improvements in correct risk evaluation compared with other visualizations. Taken together, these findings underscore that the format of SBOM presentations strongly influences not only task performance, but also risk perception.

4.3 AI Supported Risk Mitigation in Code

At the heart of this project is automating the understanding of the content of source code, both to identify code relevant to security and compliance and, equally importantly, to identify code *not* thus relevant.

In addition to traditional interactions, we will explore the potential of LLMs to support conversational agents for improve risk communication and decision-making. A chatbot could offer a more interactive and scalable solution capable of addressing concerns and answering questions regardless of the SBOM's complexity. However, the tendency of AI to provide the most common answer (e.g., "patch") versus the correct answer (e.g., "do not call this function") is a challenge to be met. Our primary goal in design of the chatbot is to provide accurate, actionable, reliable information. Previous research has found ChatGPT to be roughly 30% more accurate than Stackoverflow [28]. However, given the role of Stackoverflow in provision of inaccurate security advice ([4, 37]), this is less than optimal.

4.3.1 AI for Understanding Code

Researchers have explored the application of NLP techniques to understand and process source code, enabling more sophisticated analysis and comprehension of code semantics. Here we provide an overview, and describe previous work that supports the hypothesis embedded in our proposed research that human judgment and artificial intelligence are both required for effective risk management.

An early work in applying AI to code analysis was by Kanade et al. [61], who investigated the use of dynamic embeddings of source code by retraining CuBERT, a descendant of the BERT transformer-based NLP model, on a corpus of Python code. They evaluated the performance of CuBERT on program-understanding tasks, comparing it against other models such as Word2Vec and BiLSTM [71, 83]. The results demonstrated the potential of NLP models in improving code understanding and analysis.

Vaswani et al. [95] shifted the field of NLP by introducing the transformer-based architecture, which relies on self-attention mechanisms to capture global dependencies in text [95]. This breakthrough in NLP paved the way for subsequent advancements in code understanding and analysis. Researchers recognized the potential of the transformer-based architecture in source code analysis and applied it to the development of BERT (Bidirectional Encoder Representations from Transformers) [30]. BERT leverages self-attention mechanisms to capture contextual information and semantic relationships in natural language. Building upon this foundation, researchers introduced CodeBERT, a model pre-trained on both natural language and source code [35]. CodeBERT leverages the strengths of BERT to capture the semantic interactions and contextual understanding of code, enabling more effective analysis and comprehension.

Researchers have extended the capabilities of BERT-based models with graph neural networks, leading to the introduction of GraphCodeBERT [47]. GraphCodeBERT represents code as graphs, capturing the

hierarchical structure and relationships between different code elements. It leverages abstract syntax trees (ASTs) to encode the syntactic structure of the code, allowing for a more comprehensive understanding of code semantics. Kunz et al. [62] similarly employed code property graphs to analyze code syntactic structure, control flow, and data flow. They nevertheless did not use neural networks, but rather leveraged the LINDDUN GO [103] toolkit for privacy threat modeling to create privacy property graphs through a semi-automated process. The approach featured a manual effort in creating labels that are associated with identifiers of privacy in code.

By combining the transformer-based architecture with graph neural networks, GraphCodeBERT integrates information about the flow of data, control, and dependencies within the code. This approach enables enhanced code understanding and analysis, including tasks such as code summarization, code completion, and code similarity detection. GraphCodeBERT's utilization of ASTs and graph-based representations provides a powerful mechanism for capturing the intricacies of code and improving the accuracy of code-related tasks. The integration of these advanced models, including CuBERT, CodeBERT, and GraphCodeBERT, may improve the identification of sensitive information and privacy risks within code. By leveraging the strengths of NLP techniques, self-attention mechanisms, and transformer-based architectures, these models enable insights into code semantics.

In recent years, there have been many advancements in large language models designed for natural language processing (NLP) tasks. One of the pioneering models in this field is GPT (Generative Pre-trained Transformer) [80]. GPT leverages the transformer-based architecture which uses self-attention mechanisms to capture complex syntax and semantic relationships in textual data.

Building upon the success of GPT, researchers further developed ChatGPT, a language model designed for conversational applications [81]. ChatGPT utilizes the same transformer-based architecture as GPT but is fine-tuned for generating coherent and contextually relevant responses in conversational settings. This fine-tuning process required training the model on a large corpus of conversational data to learn patterns and structures inherent in human conversations.

Parallel to these advancements, Google developed its own conversational large language model-based chatbot that assists developers in coding called BARD. Moreover, since OpenAI's ChatGPT family of large language models are closed source and proprietary, HuggingFace is an organization focused on providing open-source pre-trained language models, libraries, and datasets. HuggingFace also released its own version called HuggingChat which HuggingFace claims is as powerful as OpenAI's models.

4.3.2 AI, Code, and Compliance

The two research tasks in this proposal are identifying vulnerable code and communicating the corresponding risk to empower harm mitigation. In this section, we describe previous work in harm mitigation. The application of AI to human interactions explaining risks in code can be informed by but are not the same as using AI to analyze code.

Large language models (LLMs) have been shown to improve efficiency in coding and identification of vulnerabilities [17, 40, 88]. Conversely, criticisms of LLMs indicate systematic patterns of failures, particularly where there are outliers [8]. These arguments offer reason to doubt that LLMs are a fit for the task of determining when a vulnerability is relevant and how to mitigate it. We propose that a combination of dynamic/static analysis, custom-trained AI, and human-centered design for developer support can leverage the role of AI by pairing AI results with effective human decision support.

Our initial investigations in using AI to detect problems in a related domain of compliance issues illustrate that this is non-trivial. In previous work, we compiled a data corpus by scraping GitHub for code that handles privacy-sensitive data. We validated all samples by human review of the code iteratively until all three researchers agreed. With the data validated, we then proceeded to zero-shot and few-shot classifiers focusing on named entity recognition, noting its failure modes. We then leveraged few-shot learning with specialized language models (CodeBERT and GraphCodeBERT) and zero-shot classification, specifically ChatGPT and

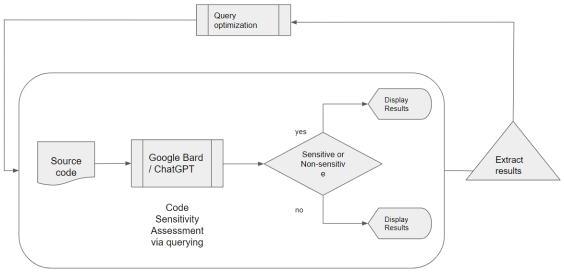

Figure 2: Zero-shot training was implemented with iterative prompt engineering

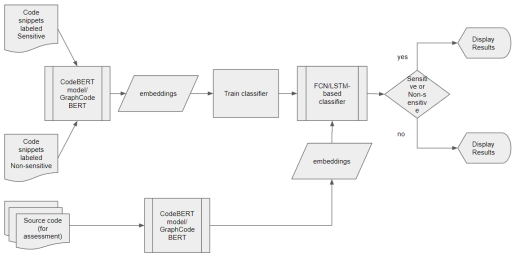

Figure 3: Few-shot learning exploits the models' pre-trained knowledge base.

BARD. Each code fragment was presented to ChatGPT 3.5, ChatGPT 4, and Gemini models using the same prompt, enabling standardized classification based on their understanding of GDPR and CCPA guidelines.

We counterbalanced our dataset by including a large number of code snippets that were not considered sensitive. This research on identification of privacy-sensitive code will inform our work on supporting the identification of code with vulnerabilities that require mitigation.

We evaluated the ability of developers to identify code, and compared this with the ability of various LLM approaches. In our proof of concept, training and testing data consisted of 57 code fragments containing sensitive data and 40 fragments without, each of which was roughly 100 lines. We evaluated ChatGPT for a baseline. We also leveraged CodeBERT and GraphCodeBERT because these language models are specifically tuned for analyzing and interpreting programming languages. We also explored the use of Long Short-Term Memory (LSTM) networks, given their superiority in processing sequence data and their ability to learn long-term dependencies [52]. Our LSTM model included two layers, each with 512 hidden units, which provided sufficient capacity to model complex dependencies in our data. Both the refined FCN and the LSTM model were implemented using the same deep-learning framework and were optimized using the Adam optimizer.

Zero-shot learning depends upon extensive pre-training on diverse data. The goal in evaluating general purpose LLMs (e.g., ChatGPT) was to leverage this capability to identify sensitive information in code without requiring the models to have extensive exposure to the specific training task. Few-shot learning exploits the models' pre-trained knowledge base which is designed to enable these models to identify patterns in sparse data and effectively adapt to novel programming tasks. Again, our proof of concepts' small sample size required efficiency with relatively smaller training samples.

We provided the same code fragments in a human subjects experiment where participants categorized code snippets according to their perception of the sensitivity of the data. In this case, the 3–10 line snippets with the potentially sensitive data were presented for sorting, with the additional lines for context beneath [68]. The patterns of errors by AI are most like those of highly skilled programmers. This suggests the potential of AI and human collaboration given the more expert programmers were much more likely to (incorrectly) mark sensitive data as not at all sensitive. This illustrates patterns across the classifications made by human and artificial intelligence.

Given long-running concerns about LLM ability to reason about code paths correctly [33, 66] rather than based on, e.g., superficial resemblance of identifiers, use of any LLM-based approaches to the relevance problem outlined above will require substantial benchmarking. Our code analysis methods can provide a critical ground-truth basis for evaluating (and training) LLM performance in real-world relevance analysis tasks, and, conversely interactive LLM approaches can potentially be made much more effective by allowing an LLM to read and analyze a large, fine-grained relevance and causality analysis report produced dynamically by hierarchical mutation methods (information often too extensive for direct examination by humans).

In principle, LLMs could even direct the use of more expensive analyses if it proves necessary to make use of coarser approximations to dynamic relevancy measures in real-world SBOMs.

5 Broader Impacts

Recent regulatory developments and industry-led initiatives underscore the growing significance of SBOMs in strengthening software supply chain security. In the United States, the Executive Order on Improving the Nation's Cybersecurity requires vendors providing software to federal agencies to produce SBOMs [50, 51], thereby setting a new benchmark for transparency and accountability. Government bodies, such as the National Institute of Standards and Technology (NIST), have further reinforced these requirements by publishing guidelines that recommend SBOM generation and maintenance as part of robust software development and risk management practices [76]. Meanwhile, collaborative efforts from the Open Source Security Foundation (OpenSSF) exemplify broader industry engagement in defining best practices, promoting interoperability, and advancing SBOM tooling [60]. At the time of this writing, ours is the only third-party comparison of SBOM visualizations, excluding claims of private vendors of usability, acceptability, and accuracy. Taken together, these mandates and collaborative initiatives not only highlight the importance of SBOMs but also facilitate a more standardized approach to secure software development and supply chain transparency.

In addition to these societal and economic benefits rooted in the core SBOM functionality and use, we expect the relevance-based analysis developed by this project to be useful in broader static analysis and program repair contexts, as discussed above. Moreover, in cases where (as in program repair and triage of static analysis results) human interaction is critical to analysis technology use, the outcomes from our investigation of how to present complex casual relevance in an SBOM context is likely to inform future human-machine partnerships in complex code analysis and development tasks, with potentially wider applicability to human-AI paired code development, a likely major thrust of all future software development.

6 Results of Prior NSF Support

PI Groce: The most relevant prior NSF support for PI Groce is CCF-2129446, "Feedback-Driven Mutation Testing for Any Language," with a total budget of $500,000 from 9/2021 until 8/2024, a collaborative proposal with Claire Le Goues of Carnegie Mellon University. *Intellectual Merit:* This project focuses on allowing developers to improve testing by using mutation testing to identify weaknesses in tests. This project has resulted in six publications [27, 44–46, 57, 58] thus far and a major software release. One of these papers [57] focuses on using increased context to improve the performance of a CodeBERT-based predictive analysis. *Broader Impact:* Work from this project has already resulted in the reporting and correction of multiple bugs in software systems, including widely used production compilers.

L. Jean Camp is current funded by a POSE: Phase I: Center for Formal and Human-Centered Security Verification in Internet of Things (IoT) Standards and Standardization (NSF 2547490, 8/2025 -8/2026, $299,929) and the POSE: Phase I: supplemental (I-Corps): (NSF 2547490, 8/2025 -8/2026, $20,000). **Intellectual Merit**: The FHS-IoT OSE delivers a continuous integration and deployment (CI/CD) pipeline for verifying IoT standards and implementations. This improves developer workflows, reduces duplication of effort, and enhances code reusability across projects. By combining formal verification methods with human-centered evaluation, it creates a unified framework that advances both technical rigor and practical usability. **Broader Impact**: The supplement is for the I-Corps program to better define broader impact. The broader impact of improved security in IoT will protect people in their homes and at work.

Her expired award was "SaTC Large: Living in the Internet of Things" (CNS 1565375, 7/16-12/21, $2.99M). **Intellectual Merit**: The research integrates usability, privacy policies and perceptions with cryptography, access control, and detection of device subversion. **Broader Impact**: We mentored over a dozen REU students (with co-authored publications, e.g. [**?**, 2, 23–25, 32, 93] two industry workshops, one education workshop, funded more than 20 hourly of graduate students, and two post-doctoral scholars. The four PIs together published over 50 peer-reviewed papers during the research.

References

[1] LLC 2025 GUAC a Series of LF Projects. Guac, 2025. `https://guac.sh/`.

[2] Jacob Abbott, Jayati Dev, Donginn Kim, Shakthidhar Gopavaram, Meera Iyer, Shivani Sadam, Shrirang Mare, Tatiana Ringenberg, Vafa Andalibi, and L Jean Camp. Privacy lessons learnt from deploying an iot ecosystem in the home. In *Proceedings of the 2022 European Symposium on Usable Security*, pages 98–110, 2022.

[3] Alain Abran, Adel Khelifi, Witold Suryn, and Ahmed Seffah. Usability meanings and interpretations in iso standards. *Software quality journal*, 11:325–338, 2003.

[4] Yasemin Acar, Michael Backes, Sascha Fahl, Doowon Kim, Michelle L Mazurek, and Christian Stransky. You get where you're looking for: The impact of information sources on code security. In *2016 IEEE symposium on security and privacy (SP)*, pages 289–305. IEEE, 2016.

[5] Yasemin Acar, Christian Stransky, Dominik Wermke, Michelle L Mazurek, and Sascha Fahl. Security developer studies with {GitHub} users: Exploring a convenience sample. In *Thirteenth Symposium on Usable Privacy and Security (SOUPS 2017)*, pages 81–95. "USENIX", 2017.

[6] Ionut Arghire. Supply chain attack: Major linux distributions impacted by xz utils backdoor, April 2024.

[7] Farnaz Behrang, Myra B Cohen, and Alessandro Orso. Users beware: Preference inconsistencies ahead. In *Proceedings of the 2015 10th Joint Meeting on Foundations of Software Engineering*, pages 295–306, 2015.

[8] Emily M. Bender, Timnit Gebru, Angelina McMillan-Major, and Shmargaret Shmitchell. On the dangers of stochastic parrots: Can language models be too big? In *Proceedings of the 2021 ACM Conference on Fairness, Accountability, and Transparency*, FAccT '21, page 610–623, New York, NY, USA, 2021. ACM.

[9] Andreas Berger. What is log4shell? the log4j vulnerability explained (and what to do about it).

[10] Al Bessey, Ken Block, Ben Chelf, Andy Chou, Bryan Fulton, Seth Hallem, Charles Henri-Gros, Asya Kamsky, Scott McPeak, and Dawson Engler. A few billion lines of code later: using static analysis to find bugs in the real world. *Commun. ACM*, 53(2):66–75, feb 2010.

[11] Eric Bodden. The secret sauce in efficient and precise static analysis: the beauty of distributive, summary-based static analyses (and how to master them). In *Companion Proceedings for the ISSTA/ECOOP 2018 Workshops*, ISSTA '18, page 85–93, New York, NY, USA, 2018. Association for Computing Machinery.

[12] L Jean Camp and Vafa Andalibi. Sbom vulnerability assessment & corresponding requirements. *NTIA Response to Notice and Request for Comments on Software Bill of Materials Elements and Considerations*, 2021.

[13] Peter Caven, Zitao Zhang, Jacob Abbott, Xinyao Ma, and Ljean Camp. Comparing the use and usefulness of four iot security labels. In *Proceedings of the 2024 CHI Conference on Human Factors in Computing Systems*, pages 1–31, 2024.

[14] Peter J Caven, Xinyao Ma, Vafa Andalibi, and L Jean Camp. Using an sbom to mitigate a lemons market. *Available at SSRN 5002288*, 2024.

[15] Anesu Chaora. SBOM dataset from 100 000+ public GitHub repositories.

[16] Anesu Chaora and L Jean Camp. Compilation of sbom dataset from 100 000+ public github repositories.

[17] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.

[18] Arpit Christi, Alex Groce, and Rahul Gopinath. Resource adaptation via test-based software minimization. In *11th IEEE International Conference on Self-Adaptive and Self-Organizing Systems, SASO 2017, Tucson, AZ, USA, September 18-22, 2017*, pages 61–70. IEEE Computer Society, 2017.

[19] CSRB. Review of the december 2021 log4j event.

[20] CVEdeatils.com. Security vulnerabilities, CVEs published in 2021 in CISA KEV catalog CVSS score between 9 and 10.

[21] Cyclonedx software bill of materials (sbom) standard. [Online]. Available on: `https://cyclonedx.org`, 2021.

[22] Gregorio Dalia, Corrado Aaron Visaggio, Andrea Di Sorbo, and Gerardo Canfora. Sbom ouverture: What we need and what we have. In *Proceedings of the 19th International Conference on Availability, Reliability and Security*, pages 1–9, 2024.

[23] Sanchari Das, Robert S Gutzwiller, Rod D Roscoe, Prashanth Rajivan, Yang Wang, L Jean Camp, and Roberto Hoyle. Humans and technology for inclusive privacy and security. In *Proceedings of the Human Factors and Ergonomics Society Annual Meeting*, volume 64, pages 461–464. SAGE Publications Sage CA: Los Angeles, CA, 2020.

[24] Sanchari Das, Andrew Kim, and L Jean Camp. Organizational security: Implementing a risk-reduction-based incentivization model for mfa adoption. In *Proceedings of the International Conference on Financial Cryptography and Data Security*, 2021.

[25] Sanchari Das, Bingxing Wang, and L Jean Camp. Mfa is a waste of time! understanding negative connotation towards mfa applications via user generated content. *arXiv preprint arXiv:1908.05902*, 2019.

[26] Fred D Davis et al. Technology acceptance model: Tam. *Al-Suqri, MN, Al-Aufi, AS: Information Seeking Behavior and Technology Adoption*, 205:219, 1989.

[27] Sourav Deb, Kush Jain, Rijnard van Tonder, Claire Le Goues, and Alex Groce. Syntax is all you need: A universal-language approach to mutant generation. In *ACM International Conference on the Foundations of Software Engineering*, 2024. Accepted for publication.

[28] Zack Delile, Sean Radel, Joe Godinez, Garrett Engstrom, Theo Brucker, Kenzie Young, and Sepideh Ghanavati. Evaluating privacy questions from stack overflow: can chatgpt compete? In *2023 IEEE 31st international requirements engineering conference workshops (REW)*, pages 239–244. IEEE, 2023.

[29] R. A. DeMillo, R. J. Lipton, and F. G. Sayward. Hints on test data selection: Help for the practicing programmer. *Computer*, 4(11), April 1978.

[30] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. 2019.

[31] Elizabeth Dinella, Gabriel Ryan, Todd Mytkowicz, and Shuvendu K. Lahiri. TOGA: A neural method for test oracle generation. In *44th IEEE/ACM 44th International Conference on Software Engineering, ICSE 2022, Pittsburgh, PA, USA, May 25-27, 2022*, pages 2130–2141. ACM, 2022.

[32] Andrew Dingman, Gianpaolo Russo, George Osterholt, Tyler Uffelman, and L. Jean Camp. Good Advice That Just Doesn't Help. In *2018 IEEE/ACM Third International Conference on Internet-of-Things Design and Implementation (IoTDI)*, pages 289–291, Orlando, FL, 2018. IEEE.

[33] Angela Fan, Beliz Gokkaya, Mark Harman, Mitya Lyubarskiy, Shubho Sengupta, Shin Yoo, and Jie M. Zhang. Large language models for software engineering: Survey and open problems. In *2023 IEEE/ACM International Conference on Software Engineering: Future of Software Engineering (ICSE-FoSE)*, pages 31–53, 2023.

[34] Adrienne Porter Felt, Erika Chin, Steve Hanna, Dawn Song, and David Wagner. Android Permissions Demystified. In *18th ACM Conference on Computer and Communications Security*, CCS '11, page 627–638, New York, NY, USA, 2011. ACM.

[35] Zhangyin Feng, Daya Guo, Duyu Tang, Nan Duan, Xiaocheng Feng, Ming Gong, Linjun Shou, Bing Qin, Ting Liu, Daxin Jiang, and Ming Zhou. CodeBERT: A Pre-Trained Model for Programming and Natural Languages. *Findings of EMNLP 2020*, page 12, September 2020.

[36] First.org. Common vulnerability scoring system version 4.0: Specification document.

[37] Felix Fischer, Konstantin Böttinger, Huang Xiao, Christian Stransky, Yasemin Acar, Michael Backes, and Sascha Fahl. Stack overflow considered harmful? the impact of copy&paste on android application security. In *2017 IEEE symposium on security and privacy (SP)*, pages 121–136. IEEE, 2017.

[38] Apache Software Foundation. Apache log4j 2.

[39] FTC. FTC warns companies to remediate log4j security vulnerability.

[40] GitHub. Github Copilot: Your AI pair programmer, 2021. Available at: `https://copilot.github.com/`.

[41] Inc GitHub.com. Exporting a software bill of materials for your repository.

[42] Matthew Green and Matthew Smith. Developers are not the enemy!: The need for usable security apis. *IEEE Security & Privacy*, 14(5):40–46, 2016.

[43] Andy Greenbeg and Matt Burgess. The mystery of 'jia tan,' the xz backdoor mastermind. *Wired*, April 2024.

[44] Alex Groce, Kush Jain, , Rijnard van Tonder, Goutamkumar Tulajappa Kalburgi, and Claire Le Goues. Looking for lacunae in bitcoin core's fuzzing efforts. In *ACM/IEEE International Conference on Software Engineering*, pages 185–186, 2022.

[45] Alex Groce, Goutamkumar Tulajappa Kalburgi, Claire Le Goues, Kush Jain, and Rahul Gopinath. First, fuzz the mutants. In *1st International Fuzzing Workshop*, 2022.

[46] Alex Groce, Rijnard van Tonder, Goutamkumar Tulajappa Kalburgi, and Claire Le Goues. Making no-fuss compiler fuzzing effective. In *ACM SIGPLAN International Conference on Compiler Construction*, pages 194–204, 2022.

[47] Daya Guo, Shuo Ren, Shuai Lu, Zhangyin Feng, Duyu Tang, Shujie Liu, Long Zhou, Nan Duan, Alexey Svyatkovskiy, Shengyu Fu, et al. GraphCodeBERT: Pre-training code representations with data flow. *International Conference on Learning Representations*, 2021.

[48] Anthony Harrison. Sbom2dot, August 2024. `https://github.com/anthonyharrison/sbom2dot`.

[49] The Health care SBOM Proof of Concept. [Online]. Available on: `https://www.ntia.doc.gov/files/ntia/publications/ntia_sbom_healthcare_poc_2020-07-09.pdf`, 2021.

[50] Paul Hernandez. Executive order 14028, improving the nation's cybersecurity, 2021. `https://www.whitehouse.gov/briefing-room/presidential-actions/2021/05/12/executive-order-on-improving\%0A-the-nations-cybersecurity`.

[51] Paul Hernandez. Executive order 14028, improving the nation's cybersecurity, 2021.

[52] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural computation*, 9(8):1735–1780, 1997.

[53] Jeffrey Hughes, Cassandra Sparks, Alley Stoughton, Rinku Parikh, Albert Reuther, and Suresh Jagannathan. Building resource adaptive software systems (brass): Objectives and system evaluation. *arXiv preprint arXiv:1510.02104*, 2015. Also published via ACM / other outlets.

[54] IBM. IBM x-force threat intelligence index 2022.

[55] 2025 Deepbits Technology Inc. Deepbits, 2025. `https://www.deepbits.com/`.

[56] Software Engineering Institute. Towards improving the CVSS.

[57] Kush Jain, Uri Alon, Alex Groce, and Claire Le Goues. Contextual predictive mutation testing. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, ESEC/FSE 2023, page 250–261, New York, NY, USA, 2023. Association for Computing Machinery.

[58] Kush Jain, Goutamkumar Tulajappa Kalburgi, Claire Le Goues, and Alex Groce. Mind the gap: The difference between coverage and mutation score can guide testing efforts. In *2023 IEEE 34th International Symposium on Software Reliability Engineering (ISSRE)*, pages 102–113, 2023.

[59] Rebecca Jones and Lucas Tate. Visualizing comparisons of bill of materials. In *2023 IEEE Symposium on Visualization for Cyber Security (VizSec)*, pages 12–16. IEEE, 2023.

[60] Josh Bressers. SBOM Everywhere and the Security Tooling Working Group: Providing the Best Security Tools for Open Source Developers, June 2023. https://openssf.org/blog/2023/06/30/sbom-everywhere-and-the-security-tooling-working-group-providing-the-best-security-tools-for-open-source-developers/.

[61] Aditya Kanade, Petros Maniatis, Gogul Balkrishnan, and Kensen Shi. Learning and Evaluating Contextual Embedding of Source Code. In *377th International Conference on Machine Learning*, volume 119. PMLR, 2020.

[62] Immanuel Kunz, Konrad Weiss, Angelika Schneider, and Christian Banse. Privacy property graph: Towards automated privacy threat modeling via static graph-based analysis. In *Proceedings on Privacy Enhancing Technologies Symposium 2023(2)*, volume 2023 of *2*, pages 171–187. Privacy Enhancing Technologies Symposium.

[63] Guilherme Lacerda, Fabio Petrillo, Marcelo Pimenta, and Yann Gaël Guéhéneuc. Code smells and refactoring: A tertiary systematic review of challenges and observations. *Journal of Systems and Software*, 167:110610, 2020.

[64] Sangchul Lee and Jae Wook Jeon. Evaluating performance of android platform using native c for embedded systems. In *ICCAS 2010*, pages 1160–1163. IEEE, 2010.

[65] Sungho Lee, Julian Dolby, and Sukyoung Ryu. Hybridroid: static analysis framework for android hybrid applications. In *2016 31st IEEE/ACM International Conference on Automated Software Engineering (ASE)*, pages 250–261. IEEE, 2016.

[66] Weixin Liang, Yuhui Zhang, Hancheng Cao, Binglu Wang, Daisy Yi Ding, Xinyu Yang, Kailas Vodrahalli, Siyu He, Daniel Scott Smith, Yian Yin, Daniel A. McFarland, and James Zou. Can large language models provide useful feedback on research papers? a large-scale empirical analysis. *NEJM AI*, 1(8):AIoa2400196, 2024.

[67] Xinyao Ma, Jacob Abbott, Zitao Zhang, and LJ Camp. Sbom: Do visualizations support decision making. In *Cyber Supply Chain Risk Management Conference*, 10 2024.

[68] Xinyao Ma, Ambarish Gurjar, Anesu Chaora, and L Jean Camp. Programmer's perception of sensitive information in code. In *USEC: Symposium on Usable Security*, 2024.

[69] Xinyao Ma, A Veldandi, Z Zhang, P Caven, A Gurjar, and LJ Camp. Poster: Usability, acceptability, and efficacy of sbom visualization and ai assistant. In *Network and Distributed System Security (NDSS) Symposium*, 2024.

[70] Michael Meli, Matthew R McNiece, and Bradley Reaves. How bad can it git? characterizing secret leakage in public github repositories. In *NDSS*, 2019.

[71] Tomas Mikolov, Kai Chen, Greg Corrado, and Jeffrey Dean. Efficient estimation of word representations in vector space. *arXiv preprint arXiv:1301.3781*, 2013.

[72] Mehdi Mirakhorli, Derek Garcia, Schuyler Dillon, Kevin Laporte, Matthew Morrison, Henry Lu, Viktoria Koscinski, and Christopher Enoch. A landscape study of open source and proprietary tools for software bill of materials (sbom). *arXiv preprint arXiv:2402.11151*, 2024.

[73] Ghassan Misherghi and Zhendong Su. Hdd: hierarchical delta debugging. In *International Conference on Software engineering*, pages 142–151, 2006.

[74] MITRE. About the CVE program.

[75] National Telecommunications and Information Administrator. Framing Software Component Transparency: Establishing a Common Software Bill of Materials (SBOM). 2020.

[76] Michael Nieles, Kelley Dempsey, and Victoria Yan Pillitteri. An Introduction to Information Security. Technical Report NIST Special Publication 800-12 Revision 1, U.S. Department of Commerce, June 2017.

[77] Jakob Nielsen. Enhancing the explanatory power of usability heuristics. In *Proceedings of the SIGCHI conference on Human Factors in Computing Systems*, pages 152–158, 1994.

[78] J. Offutt and A. Abdurazik. In *Mutation 2000: Mutation Testing in the Twentieth and the Twenty First Centuries*, 2000.

[79] Chenxiong Qian, Xiapu Luo, Yuru Shao, and Alvin TS Chan. On tracking information flows through jni in android applications. In *2014 44th Annual IEEE/IFIP International Conference on Dependable Systems and Networks*, pages 180–191. IEEE, 2014.

[80] Alec Radford, Karthik Narasimhan, Tim Salimans, Ilya Sutskever, et al. Improving Language Understanding by Generative Pre-Training. 2018.

[81] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, Ilya Sutskever, et al. Language models are unsupervised multitask learners. *OpenAI blog*, 1(8):9, 2019.

[82] Hocine Rebatchi, Tégawendé F Bissyandé, and Naouel Moha. Dependabot and security pull requests: large empirical study. *Empirical Software Engineering*, 29(5):128, 2024.

[83] M. Schuster and K.K. Paliwal. Bidirectional recurrent neural networks. *IEEE Transactions on Signal Processing*, 45(11):2673–2681, 1997.

[84] Mohammed Latif Siddiq, Joanna C. S. Santos, Ridwanul Hasan Tanvir, Noshin Ulfat, Fahmid Al Rifat, and Vinicius Carvalho Lopes. An empirical study of using large language models for unit test generation, 2023.

[85] slashdot. *aws* urges devs to scrub secret keys from github, 2023.

[86] The software package data exchange (spdx). [Online]. Available on: `https://spdx.dev`, 2021.

[87] Trevor Stalnaker, Nathan Wintersgill, Oscar Chaparro, Massimiliano Di Penta, Daniel M German, and Denys Poshyvanyk. Boms away! inside the minds of stakeholders: A comprehensive study of bills of materials for software systems. 2024.

[88] Alexey Svyatkovskiy, Shao Kun Deng, Shengyu Fu, and Neel Sundaresan. Intellicode compose: Code generation using transformer. In *Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, ESEC/FSE 2020, page 1433–1443, New York, NY, USA, 2020. ACM.

[89] Mohammad Tahaei and Kami Vaniea. "developers are responsible": What ad networks tell developers about privacy. In *Extended Abstracts of the 2021 CHI Conference on Human Factors in Computing Systems*, pages 1–11, 2021.

[90] National Telecommunications and Information Administration. Software Suppliers Playbook: SBOM Production and Provision. 2021. `https://www.ntia.doc.gov/report/2021/minimum-elements-software-bill-materials-sbom`.

[91] Spdx software bill of materials (SBOM) generator. [Online]. Available on: `https://github.com/spdx/spdx-sbom-generator`, 2021.

[92] TrailofBits. Itdepends, December 2024. `https://github.com/trailofbits/it-depends`.

[93] Ploy Unchit, Sanchari Das, Andrew Kim, and L Jean Camp. Quantifying susceptibility to spear phishing in a high school environment using signal detection theory. In *International Symposium on Human Aspects of Information Security and Assurance*, pages 109–120. Springer, 2020.

[94] Paul Van Schaik, Debora Jeske, Joseph Onibokun, Lynne Coventry, Jurjen Jansen, and Petko Kusev. Risk perceptions of cyber-security and precautionary behaviour. *Computers in Human Behavior*, 75:547–559, 2017.

[95] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in Neural Information Processing Systems*, 30, 2017.

[96] Oscar Luis Vera-Pérez, Benjamin Danglot, Martin Monperrus, and Benoit Baudry. A comprehensive study of pseudo-tested methods. *Empirical Softw. Engg.*, 24(3):1195–1225, June 2019.

[97] Slashdot Website. Github kills search after hundreds of private keys exposed, 2013.

[98] Chamila Wijayarathna and Nalin Asanka Gamagedara Arachchilage. Am i responsible for end-user's security? a programmer's perspective. In *4th Workshop on Security Information Workers*, 2018.

[99] Laurie Williams, Giacomo Benedetti, Sivana Hamer, Ranindya Paramitha, Imranur Rahman, Mahzabin Tamanna, Greg Tystahl, Nusrat Zahan, Patrick Morrison, Yasemin Acar, et al. Research directions in software supply chain security. *ACM Transactions on Software Engineering and Methodology*, 2024.

[100] Michael D Williams, Nripendra P Rana, and Yogesh K Dwivedi. The unified theory of acceptance and use of technology (utaut): a literature review. *Journal of enterprise information management*, 28(3):443–488, 2015.

[101] Wei Wu, Pu Wang, Lei Zhao, and Wei Jiang. An intelligent security detection and response scheme based on sbom for securing iot terminal devices. In *2023 IEEE 11th International Conference on Information, Communication and Networks (ICICN)*, pages 391–398. IEEE, 2023.

[102] Julia Wunder, Andreas Kurtz, Christian Eichenmüller, Freya Gassmann, and Zinaida Benenson. Shedding light on cvss scoring inconsistencies: A user-centric study on evaluating widespread security vulnerabilities. *45th IEEE Symposium on Security and Privacy*, 2024.

[103] Kim Wuyts, Laurens Sion, and Wouter Joosen. LINDDUN GO: A light weight approach to privacy threat modeling. In *IEEE European Symposium on Security and Privacy Workshops (EuroS&PW)*, pages 302–309. IEEE.

[104] Zhiqiang Yuan, Yiling Lou, Mingwei Liu, Shiji Ding, Kaixin Wang, Yixuan Chen, and Xin Peng. No more manual tests? evaluating and improving chatgpt for unit test generation, 2023.

[105] Yunze Zhao, Yuchen Zhang, Dan Chacko, and Justin Cappos. CovSBOM: Enhancing Software Bill of Materials with Integrated Code Coverage Analysis . In *2024 IEEE 35th International Symposium on Software Reliability Engineering (ISSRE)*, pages 228–237, Los Alamitos, CA, USA, October 2024. IEEE Computer Society.